四足机器狗导航 SDK 接口规格书

本文档为 Quad SDK 公开接口的完整规格说明，接口定义以 `src/sdk/facade/quad/include/robot/quad` 下当前公开头文件为准。

目录

- 1. 概述
- 2. 命名空间与头文件
- 3. 主入口：Robot
- 4. 导航业务域：nav::Domain
- 5. 导航控制模块：nav::NavigationModule
- 6. 定位模块：nav::MappingModule
- 7. 资源模块：nav::ResourceModule
- 8. 类型与版本定义参考
- 附录：类型索引

---

1. 概述

Quad SDK 是四足机器人面向用户的抽象层，提供：

- 机器人连接与心跳管理
- 运动模式切换
- 定点导航与循迹导航
- 定位（SLAM）控制与状态查询
- 地图资源（场景、路径）同步、导出与管理
- 统一错误码与日志输出

典型使用流程：

```
1. 构造 Robot 并指定 IP
2. connect(timeout) 建立连接
3. nav().resource().refreshMapResources() 同步地图资源
4. nav().mapping().startLocalization() 或 startLocalization(scene_name, initial_pose) 启动定位 / 重定位
5. nav().navigation().setMotionMode() 设置运动模式
6. nav().navigation().startNavigation() 发起导航
7. 通过 getNavigationState() / getPauseState() 读取状态；需要事件驱动时再使用订阅接口
```

简单接入建议：

- 控制类接口如 `refreshMapResources()`、`startLocalization()`、`setMotionMode()`、`startNavigation()`、`setNavigationPaused()`、`stopNavigation()` 都是异步接口；示例中通常会用 `std::promise` / `std::future` 把回调结果转成同步等待，便于顺序演示接口调用流程。

- 对于示例程序、调试工具或简单业务，优先使用 `getLocalizationState()`、`getNavigationState()`、`getPauseState()` 直接读取缓存即可，不必一开始就接入事件订阅。
- 当业务需要实时 UI 刷新、后台事件流或长期监听时，再考虑 `registerLocalizationStateEvent()`、`registerNavigationStateEvent()`、`registerNavigationResultEvent()`、`registerPauseStateEvent()` 这类订阅接口。

---

2. 命名空间与头文件

头文件	内容
`robot/quad/api.hpp`	聚合头，统一导出公开类型、版本、Robot 与导航业务域接口；建议业务工程直接包含此文件
`robot/quad/types.hpp`	公开类型聚合头，统一导出 common、motion、nav 与 robot 领域类型
`robot/quad/version.hpp`	`robot::quad`（SDK 版本信息）
`robot/quad/robot.hpp`	`robot::quad::Robot`
`robot/quad/nav/domain.hpp`	`robot::quad::nav::Domain`
`robot/quad/nav/navigation_module.hpp`	`robot::quad::nav::NavigationModule`
`robot/quad/nav/mapping_module.hpp`	`robot::quad::nav::MappingModule`
`robot/quad/nav/resource_module.hpp`	`robot::quad::nav::ResourceModule`
`robot/quad/common/types.hpp`	`robot::quad`（通用类型）
`robot/quad/error.hpp`	`robot::quad`（错误码）
`robot/quad/nav/types.hpp`	`robot::quad`（导航、定位与地图资源类型）
`robot/quad/motion/types.hpp`	`robot::quad`（运动类型）
`robot/quad/robot/types.hpp`	`robot::quad`（机器人类型）

---

3. 主入口：Robot

头文件: `robot/quad/robot.hpp`

简要说明: `Robot` 是 Quad SDK 的主入口。典型使用流程：构造并指定 IP → `connect(timeout)` 建立连接 → 通过 `nav()` 访问资源、定位、导航模块。日志配置采用全局静态接口，对当前进程内全部 Quad SDK 实例生效。

3.1 构造函数

```
explicit Robot(const RobotOptions& options = RobotOptions{})
```

构造机器人对象。

参数	类型	说明
`options`	RobotOptions	机器人连接参数

3.2 连接管理

`bool connect(std::chrono::milliseconds timeout)`

在给定超时内同步建立连接并等待机器人首个心跳。

参数:

参数	类型	说明
`timeout`	`std::chrono::milliseconds`	等待连接就绪的最长时长

返回值: 在超时前进入就绪状态返回 `true`，否则返回 `false`

说明: 成功条件为 MQTT 传输层已连接、必需订阅已建立，并已收到当前连接周期的首个心跳。若在超时前未进入就绪状态，则本次连接尝试会被主动清理并返回失败。

`void disconnect()`

断开与机器人的连接。

`bool isConnected() const`

查询 SDK 当前是否处于连接状态。

返回值: `true` 表示连接已建立

`bool isRobotOnline() const`

查询机器人在线状态缓存。

返回值: `true` 表示机器人心跳正常

`std::string getRobotIp() const`

获取当前机器人 IP。

返回值: 机器人 IP 字符串

3.3 导航业务域

`nav::Domain& nav()`

访问导航业务域。通过该入口访问 `resource()`、`mapping()` 和 `navigation()` 三个子模块。

返回值: nav::Domain 引用

3.4 日志配置（静态全局）

方法	参数	返回值	说明
`setLogLevel`	`level`：LogLevel	`void`	同时设置控制台与文件日志等级
`setConsoleLogLevel`	`level`：LogLevel	`void`	单独设置控制台日志等级（默认 WARN）
`setFileLogLevel`	`level`：LogLevel	`void`	单独设置文件日志等级（默认 DEBUG）
`setLogFilePath`	`file_path`：`const std::string&`	`void`	设置日志文件路径（默认 `./logs/quad_sdk.log`）
`setLogCallback`	`callback`：LogCallback	`void`	注册全局日志回调

环境变量覆盖（仅在未显式配置时生效）：`ROBOT_QUAD_LOG_LEVEL` 可同时覆盖控制台与文件日志等级；`ROBOT_QUAD_CONSOLE_LOG_LEVEL`、`ROBOT_QUAD_FILE_LOG_LEVEL` 可分别覆盖控制台与文件日志等级。

---

4. 导航业务域：nav::Domain

头文件: `robot/quad/nav/domain.hpp`

简要说明: `Domain` 把 Quad SDK 对外暴露的能力收敛为三类：`navigation()` 运动模式与导航控制；`mapping()` 定位控制与状态读取；`resource()` 地图资源同步、查询与导出。

方法	返回类型	说明
`navigation()`	NavigationModule&	运动模式与导航控制
`mapping()`	MappingModule&	定位控制与状态读取
`resource()`	ResourceModule&	地图资源同步、查询与导出

---

5. 导航控制模块：nav::NavigationModule

头文件: `robot/quad/nav/navigation_module.hpp`

简要说明: 该模块负责运动模式切换、定点/循迹导航、暂停/恢复与导航状态订阅。公开控制接口均为异步接口：函数返回时仅表示请求已提交，实际受理结果通过 AsyncCommandCallback 返回。对简单业务可优先使用 `getNavigationState()` 与 `getPauseState()` 读取缓存；需要事件驱动时再使用订阅接口。

5.1 运动模式

`AsyncCommandId setMotionMode(MotionMode mode, AsyncCommandCallback callback = {})`

切换机器人运动模式。

参数	类型	说明
`mode`	MotionMode	目标运动模式
`callback`	AsyncCommandCallback	异步命令回调；回调结果表示命令受理状态

返回值: AsyncCommandId，非零表示请求已受理，0 表示请求未入队

`MotionMode getMotionMode() const`

获取当前运动模式缓存。

返回值: MotionMode

5.2 定点导航

`AsyncCommandId startNavigation(const std::string& scene_name, const Pose& target, AsyncCommandCallback callback = {})`

发起定点导航。

参数	类型	说明
`scene_name`	`const std::string&`	目标场景名称
`target`	Pose	目标位姿（位置与四元数朝向）
`callback`	AsyncCommandCallback	异步命令回调

返回值: AsyncCommandId

说明: 当前定点导航仅支持二维平面，实际只使用 `target.position.x`、`target.position.y` 与绕 Z 轴偏航角。可配合 pose2DFromPose()、poseFromPose2D() 或 poseFromXYYawDegrees() 使用。同一 SDK 实例同一时间只允许一个导航任务处于 active 生命周期；在上一任务到达终态或 `stopNavigation()` 完成前再次调用 `startNavigation()`，回调会返回 `SDK_NAVIGATION_BUSY`，且 SDK 不会下发新的导航命令。

5.3 循迹导航

`AsyncCommandId startNavigation(const std::string& scene_name, int32_t path_id, bool forward = true, uint32_t obstacle_timeout_s = 30, AsyncCommandCallback callback = {})`

发起循迹导航。

参数	类型	说明
`scene_name`	`const std::string&`	路径所属场景名称
`path_id`	`int32_t`	轨迹路径 ID，可从 MapResourceSnapshot::trajectories 中各 `NavigationTrajectory::paths` 的 `NavigationPath::path_id` 获取
`forward`	`bool`	`true` 表示正向循迹，`false` 表示反向循迹
`obstacle_timeout_s`	`uint32_t`	障碍等待超时时间，单位为秒
`callback`	AsyncCommandCallback	异步命令回调

返回值: AsyncCommandId

说明: 与定点导航相同，循迹导航受 SDK 导航 active 门禁保护；上一导航任务未结束前再次启动会返回 `SDK_NAVIGATION_BUSY`。

5.4 停止与暂停

`AsyncCommandId stopNavigation(AsyncCommandCallback callback = {})`

停止当前导航任务。

参数	类型	说明
`callback`	AsyncCommandCallback	异步命令回调

返回值: AsyncCommandId

说明: `stopNavigation()` 下发后，SDK 会继续保持导航 active，直到收到 `CANCELLED`、`IDLE`、`FINISHED` 或 `FAILED` 等停止/终态反馈后才允许新的 `startNavigation()`。

`AsyncCommandId setNavigationPaused(bool paused, AsyncCommandCallback callback = {})`

设置导航暂停或恢复。

参数	类型	说明
`paused`	`bool`	`true` 表示暂停，`false` 表示恢复
`callback`	AsyncCommandCallback	异步命令回调

返回值: AsyncCommandId

`PauseState getPauseState() const`

获取当前导航暂停状态缓存。

返回值: PauseState

说明: 对于示例程序、调试工具或简单轮询业务，建议优先使用该接口直接读取暂停状态；只有在需要实时事件推送时再使用 `registerPauseStateEvent()`。

5.5 状态订阅

`Subscription registerPauseStateEvent(PauseStateEventCallback callback, SubscribeOptions options = {})`

订阅导航暂停状态。

参数	类型	说明
`callback`	PauseStateEventCallback	暂停状态回调
`options`	SubscribeOptions	订阅选项

返回值: Subscription，析构或调用 `unsubscribe()` 后停止接收回调

订阅选项说明:

- `replay_latest`：注册后先回放当前缓存
- `only_changed`：仅在状态变化时回调
- `fresh_only`：忽略非新鲜缓存

`Subscription registerNavigationStateEvent(NavigationStateEventCallback callback, SubscribeOptions options = {})`

订阅导航状态变化。

参数	类型	说明
`callback`	NavigationStateEventCallback	导航状态回调
`options`	SubscribeOptions	订阅选项

返回值: Subscription

`Subscription registerNavigationResultEvent(NavigationResultEventCallback callback, SubscribeOptions options = {})`

订阅导航终态结果事件。

参数	类型	说明
`callback`	NavigationResultEventCallback	导航结果回调
`options`	SubscribeOptions	订阅选项

返回值: Subscription

5.6 状态读取

`NavigationState getNavigationState() const`

获取当前导航状态缓存。

返回值: NavigationState

说明:

- `error` 为 SDK 映射后的统一错误码，`error_message` 保留原始描述信息便于排障。
- 对于简单接入，推荐直接轮询该接口读取 `status`、`task_id`、`error`、`output_status`。
- 常见轮询用途包括：判断导航是否进入终态、检查是否处于停障状态、在 `stopNavigation()` 后等待最终状态落稳。

`NavigationResult getNavigationResult() const`

获取最近一次导航终态结果缓存。

返回值: NavigationResult

---

6. 定位模块：nav::MappingModule

头文件: `robot/quad/nav/mapping_module.hpp`

简要说明: 出于目录稳定性与历史命名兼容，类名仍沿用 `MappingModule`。在当前 Quad SDK 中，该模块对外提供定位控制、状态读取与状态订阅能力。

6.1 定位控制

`AsyncCommandId startLocalization(const std::string& scene_name, AsyncCommandCallback callback = {})`

基于指定场景启动定位。

参数	类型	说明
`scene_name`	`const std::string&`	场景名称，建议先通过 `resource().refreshMapResources()` 刷新资源缓存
`callback`	AsyncCommandCallback	异步命令回调

返回值: AsyncCommandId

说明: 当前 Quad 示例中，直接以该异步回调结果作为定位启动结果判断。如业务还需要读取定位缓存，可继续调用 `getLocalizationState()`。

`AsyncCommandId startLocalization(const std::string& scene_name, const Pose& initial_pose, AsyncCommandCallback callback = {})`

基于指定场景和初始位姿启动定位。

参数	类型	说明
`scene_name`	`const std::string&`	场景名称，建议先通过 `resource().refreshMapResources()` 刷新资源缓存
`initial_pose`	Pose	提供给定位模块的初始位姿
`callback`	AsyncCommandCallback	异步命令回调

返回值: AsyncCommandId

说明:

- 该接口适合已知机器人大致位置的场景，也就是重定位。
- `initial_pose` 使用 Pose 表达；如业务只关心二维平面位姿，可配合 poseFromPose2D() 或 poseFromXYYawDegrees() 构造。
- 该接口回调成功只表示命令受理成功；机器人真正完成重定位后，通常还需要继续轮询 `getLocalizationState()`，等待 `ready=true` 且 `fresh=true`。

简单调用示例:

```
std::promise<robot::quad::ErrorCode> relocalization_result_promise;
auto relocalization_result_future = relocalization_result_promise.get_future();

robot.nav().mapping().startLocalization(
    scene_name,
    initial_pose,
    [&relocalization_result_promise](const robot::quad::AsyncCommandResult& result) {
        relocalization_result_promise.set_value(result.error);
    });

if (relocalization_result_future.get() != robot::quad::ErrorCode::SUCCESS) {
    std::cerr << "启动重定位失败\n";
}
```

```
while (true) {
    const auto state = robot.nav().mapping().getLocalizationState();
    if (state.ready && state.fresh) {
        break;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(200));
}
```

`AsyncCommandId stopLocalization(const std::string& scene_name, AsyncCommandCallback callback = {})`

停止当前定位任务。

参数	类型	说明
`scene_name`	`const std::string&`	当前定位所对应的场景名称
`callback`	AsyncCommandCallback	异步命令回调

返回值: AsyncCommandId

6.2 状态读取

`LocalizationState getLocalizationState() const`

获取当前定位状态缓存。

返回值: LocalizationState

说明: 常见读取字段包括 `ready`、`fresh`、`pose.position.x / pose.position.y` 以及 `laser_quality`。对简单示例而言，启动定位或重定位后直接轮询该接口即可。

6.3 状态订阅

`Subscription registerLocalizationStateEvent(LocalizationStateEventCallback callback, SubscribeOptions options = {})`

订阅定位状态变化。

参数	类型	说明
`callback`	LocalizationStateEventCallback	定位状态回调
`options`	SubscribeOptions	订阅选项：`replay_latest` 表示注册后先回放当前缓存，`only_changed` 表示仅在状态变化时回调，`fresh_only` 表示忽略非新鲜缓存

返回值: Subscription

说明:

- 当定位缓存更新时，SDK 会重新计算 `LocalizationState` 并通过该接口分发。
- 该接口适合实时 UI 刷新、长期监控或需要订阅位姿变化的场景。
- 若只需在定位开始后等待状态就绪，继续轮询 `getLocalizationState()` 仍然是更简单的接入方式。

---

7. 资源模块：nav::ResourceModule

头文件: `robot/quad/nav/resource_module.hpp`

简要说明: 该模块负责地图资源缓存刷新、资源缓存读取、地图导出，以及场景/路径管理。

7.1 资源同步

`AsyncCommandId refreshMapResources(AsyncCommandCallback callback = {})`

从机器人拉取场景与轨迹资源，并刷新 SDK 本地缓存。

参数	类型	说明
`callback`	AsyncCommandCallback	异步命令回调

返回值: AsyncCommandId

说明: 命令回调后仍建议检查 `getMapResources()` 返回值中的 `scenes_valid` 与 `trajectories_valid` 字段，确认缓存可用。

`MapResourceSnapshot getMapResources() const`

获取当前地图资源缓存快照。

返回值: MapResourceSnapshot

7.2 路径管理

`AsyncCommandId renamePath(const std::string& scene_name, const std::string& old_name, const std::string& new_name, AsyncCommandCallback callback = {})`

重命名指定路径。

参数	类型	说明
`scene_name`	`const std::string&`	路径所属场景名称
`old_name`	`const std::string&`	原路径名称
`new_name`	`const std::string&`	新路径名称
`callback`	AsyncCommandCallback	异步命令回调

返回值: AsyncCommandId

7.3 地图导出

`void exportMapToLocal(const std::string& scene_name, uint32_t sub_scene_id, const std::string& save_dir, MapTransferCallback callback, MapTransferOptions options = {})`

导出指定子场景地图到本地目录。

参数	类型	说明
`scene_name`	`const std::string&`	场景名称
`sub_scene_id`	`uint32_t`	子场景 ID
`save_dir`	`const std::string&`	本地保存目录
`callback`	MapTransferCallback	导出结果回调
`options`	MapTransferOptions	导出超时参数

7.4 事件订阅

`Subscription registerMapResourceEvent(MapResourceEventCallback callback, SubscribeOptions options = {})`

订阅地图资源事件。

参数	类型	说明
`callback`	MapResourceEventCallback	资源事件回调
`options`	SubscribeOptions	订阅选项

返回值: Subscription

---

8. 类型与版本定义参考

8.1 通用类型（common/types.hpp）

Point3D

三维点坐标。

```
struct Point3D {
    float x{0};    // X 坐标
    float y{0};    // Y 坐标
    float z{0};    // Z 坐标
};
```

Quaternion

四元数姿态。

```
struct Quaternion {
    float x{0};    // x 分量
    float y{0};    // y 分量
    float z{0};    // z 分量
    float w{1};    // w 分量
};
```

Pose

位置与朝向组成的位姿。

```
struct Pose {
    Point3D    position;       // 位置
    Quaternion orientation;    // 姿态
};
```

Pose2D

二维平面位姿。

```
struct Pose2D {
    float x{0};             // X 坐标
    float y{0};             // Y 坐标
    float yaw_degree{0};    // 绕 Z 轴的偏航角（度）
};
```

位姿工具函数

`pose2DFromPose(const Pose& pose) -> Pose2D` **{#pose2dfrompose}**

从三维位姿中提取二维平面位姿。偏航角范围归一化到 `[-180, 180)`。

```
inline Pose2D pose2DFromPose(const Pose& pose);
```

`poseFromPose2D(const Pose2D& pose_2d, float z = 0.0f) -> Pose` **{#posefrompose2d}**

由二维平面位姿构造 `Pose`。适用于当前仅支持二维平面的定点导航接口。

```
inline Pose poseFromPose2D(const Pose2D& pose_2d, float z = 0.0f);
```

`poseFromXYYawDegrees(float x, float y, float yaw_degree, float z = 0.0f) -> Pose` **{#posefromxyyawdegrees}**

由 `x`、`y` 与偏航角快速构造 `Pose`。

```
inline Pose poseFromXYYawDegrees(float x, float y, float yaw_degree, float z = 0.0f);
```

Subscription {#subscription}

订阅生命周期句柄。析构、移动赋值或显式调用 `unsubscribe()` 时触发清理逻辑。

```
class Subscription {
public:
    Subscription() = default;
    ~Subscription();
    Subscription(const Subscription&) = delete;
    Subscription& operator=(const Subscription&) = delete;
    Subscription(Subscription&& other) noexcept;
    Subscription& operator=(Subscription&& other) noexcept;
    void unsubscribe();     // 主动退订，重复调用安全
    bool valid() const;     // 返回是否仍绑定到有效订阅
};
```

SubscribeOptions {#subscribeoptions}

订阅行为选项。

```
struct SubscribeOptions {
    bool replay_latest{true};    // 订阅建立后是否立即回放最近一次缓存值
    bool only_changed{false};    // 是否仅在内容发生变化时触发回调
    bool fresh_only{false};      // 是否仅接收当前连接周期内的新鲜数据
};
```

AsyncCommandId {#asynccommandid}

异步命令 ID。

```
using AsyncCommandId = uint64_t;
```

AsyncCommandStatus {#asynccommandstatus}

异步命令执行状态。

```
enum class AsyncCommandStatus {
    SUCCEEDED = 0,       // 命令成功完成
    TIMEOUT = 1,         // 命令等待超时
    PUBLISH_FAILED = 2,  // 命令发布失败
    CANCELLED = 3,       // 命令被取消
    SHUTDOWN = 4,        // SDK 运行时已关闭
    FAILED = 5,          // 命令执行失败
};
```

AsyncCommandResult {#asynccommandresult}

异步命令执行结果。

```
struct AsyncCommandResult {
    AsyncCommandId     command_id{0};
    AsyncCommandStatus status{AsyncCommandStatus::FAILED};
    ErrorCode          error{ErrorCode::SUCCESS};
    std::string        command_name;
    std::string        detail;
};
```

AsyncCommandCallback {#asynccommandcallback}

异步命令结果回调。

```
using AsyncCommandCallback = std::function<void(const AsyncCommandResult&)>;
```

8.2 错误类型（error.hpp）

ErrorCode {#errorcode}

统一错误码。

```
enum class ErrorCode {
    SUCCESS = 0,

    // 导航错误 (10-99)
    NAV_UNKNOWN_TASK = 10,
    NAV_STOP_BARRIER = 11,
    NAV_MAP_CHANGE_FAIL = 12,
    NAV_MAP_INDEX_ERROR = 13,
    NAV_STOP_BARRIER_ON_STAIR = 14,
    NAV_PREEMPTED = 20,
    NAV_TASK_CANCELLED = 21,
    NAV_UNKNOWN_ERROR = 99,
    NAV_OUT_OF_TRACK = 30,
    NAV_NAV_COLLAPSE = 31,
    NAV_FAR_AWAY_PATH = 32,
    NAV_UWB_DATA_GET_SUCCEED = 33,
    NAV_UWB_DATA_GET_FAIL = 34,
    NAV_TF_ERROR = 35,
    NAV_PAT_EXCEEDED = 36,
    NAV_TASK_PATH_INVALID = 37,

    // SLAM 错误 (300-399)
    SLAM_NORMAL = 300,
    SLAM_UNABLE_START = 301,
    SLAM_NO_IMU = 302,
    SLAM_NO_LIDAR = 303,
    SLAM_LIDAR_INCOMPLETE = 304,
    SLAM_MAP_NOFOUND = 305,
    SLAM_LOCALIZE_ERROR = 306,
    SLAM_COLLAPSE = 307,
    SLAM_MAPPING_FAIL = 308,
```

```
    SLAM_LIDAR_ERROR = 309,
    SLAM_IMU_ERROR = 310,
    SLAM_GNSS_ERROR = 311,
    SLAM_SYNC_ERROR = 312,

    // SDK 运行时错误 (600-699)
    SDK_COMMAND_TIMEOUT = 600,
    SDK_MQTT_PUBLISH_FAILED = 601,
    SDK_TRANSPORT_DISCONNECTED = 602,
    SDK_ROBOT_OFFLINE = 603,
    SDK_COMMAND_CANCELLED = 604,
    SDK_RUNTIME_SHUTDOWN = 605,
    SDK_COMMAND_FAILED = 606,
    SDK_RESOURCE_NOT_FOUND = 607,
    SDK_IO_ERROR = 608,
    SDK_PROTOCOL_UNKNOWN_CODE = 609,
    SDK_COMMAND_SUPERSEDED = 610,
    SDK_NAVIGATION_BUSY = 611,
    SDK_INTERNAL_ERROR = 699,
};
```

ErrorItem {#erroritem}

错误详情。

```
struct ErrorItem {
    ErrorCode   code{ErrorCode::SUCCESS};
    std::string name;
    std::string description_en;
    std::string description_cn;
    std::string detail;
};
```

ErrorDescriptionLanguage {#errordescriptionlanguage}

错误描述语言。

```
enum class ErrorDescriptionLanguage {
    En,    // 英文描述
    Zh,    // 中文描述
};
```

`formatErrorCodeForLog(ErrorCode code, ErrorDescriptionLanguage lang = En) -> std::string`

以日志友好格式输出错误码。输出格式：`< ErrorCode | NAME | DESCRIPTION >`。

```
ROBOT_QUAD_API std::string formatErrorCodeForLog(
    ErrorCode code,
    ErrorDescriptionLanguage lang = ErrorDescriptionLanguage::En);
```

8.3 导航、定位与地图资源类型（nav/types.hpp）

NavigationStatus {#navigationstatus}

导航任务状态。

```
enum class NavigationStatus {
    IDLE = 0,       // 空闲
    RUNNING = 1,    // 执行中
    FINISHED = 2,   // 已完成
    CANCELLED = 3, // 已取消
    FAILED = 4      // 执行失败
};
```

MapResourceEventType {#mapresourceeventtype}

地图资源事件类型。

```
enum class MapResourceEventType {
    SCENES_UPDATED = 0,
    TRAJECTORIES_UPDATED = 1,
    SCENE_REMOVED = 2,
    TRAJECTORY_REMOVED = 3,
    PATH_REMOVED = 4,
    PATH_RENAMED = 5,
};
```

MapTransferOptions {#maptransferoptions}

地图文件传输选项。

```
struct MapTransferOptions {
    std::chrono::seconds transfer_timeout{60};   // 文件传输总超时
    std::chrono::seconds connect_timeout{10};    // FTP 连接超时
};
```

NavigationTaskMode {#navigationtaskmode}

导航任务模式。

```
enum class NavigationTaskMode : uint32_t {
    POSE = 0,              // 定点导航
    PATH = 1,              // 循迹导航
    UNKNOWN = 0xFFFFFFFF   // 未知模式
};
```

NavigationOutputStatus {#navigationoutputstatus}

导航输出状态。

```
enum class NavigationOutputStatus : uint32_t {
    NONE = 0,              // 无任务状态
    FINISHED = 1,          // 导航任务完成
    RUNNING = 2,           // 导航任务正在执行
    CANCELLED = 3,         // 导航任务取消
    FAILED = 4,            // 导航任务失败
    UNKNOWN = 0xFFFFFFFF   // 未知输出状态
};
```

NavigationState {#navigationstate}

当前导航状态快照。

```
struct NavigationState {
    NavigationStatus       status{NavigationStatus::IDLE};
    uint32_t               task_id{0};
    NavigationTaskMode     task_mode{NavigationTaskMode::UNKNOWN};
    NavigationOutputStatus output_status{NavigationOutputStatus::NONE};
    uint32_t               current_map_index{0};
    ErrorCode              error{ErrorCode::SUCCESS};
    std::string            error_message;
    bool                   fresh{false};  // 当前连接周期内是否已收到有效上报
};
```

NavigationResult {#navigationresult}

最近一次导航终态结果。

```
struct NavigationResult {
    NavigationStatus status{NavigationStatus::IDLE};
    ErrorCode        error{ErrorCode::SUCCESS};
    bool             fresh{false};
};
```

PauseState {#pausestate}

导航暂停状态。

```
struct PauseState {
    bool paused{false};
    bool fresh{false};
};
```

SceneInfo {#sceneinfo}

子场景信息。

```
struct SceneInfo {
    uint32_t    sub_scene_id{0};
    std::string yam_filename;
    std::string pgm_filename;
};
```

SceneDetail {#scenedetail}

场景详情。

```
struct SceneDetail {
    uint32_t               scene_id{0};
    std::string            scene_name;
    std::vector<SceneInfo> sub_scenes;
};
```

NavigationPoint {#navigationpoint}

导航路点。

```
struct NavigationPoint {
    int32_t point_id{0};
    int32_t sub_scene_id{0};
    Pose    pose;
};
```

NavigationPath {#navigationpath}

导航路径。

```
struct NavigationPath {
    int32_t           path_id{0};
    std::string       path_name;
    int32_t           start_point_id{0};
    int32_t           end_point_id{0};
    std::vector<Pose> points;
};
```

NavigationTrajectory {#navigationtrajectory}

场景中的导航拓扑。

```cpp
struct NavigationTrajectory {
    int32_t                      graph_id{0};
    std::string                  scene_name;
    std::vector<NavigationPoint> waypoints;
    std::vector<NavigationPath>  paths;
};
```

LocalizationState {#localizationstate}

定位状态快照。

```cpp
struct LocalizationState {
    Pose  pose;
    float laser_quality{0};
    bool  ready{false};
    bool  fresh{false};
};
```

MapResourceSnapshot {#mapresourcesnapshot}

地图资源缓存快照。

```cpp
struct MapResourceSnapshot {
    std::vector<SceneDetail>          scenes;
    std::vector<NavigationTrajectory> trajectories;
    bool                              scenes_valid{false};
    bool                              trajectories_valid{false};
    uint64_t                          scene_resource_version{0};
    uint64_t                          trajectory_resource_version{0};
    uint64_t                          connection_generation{0};
};
```

MapResourceEvent {#mapresourceevent}

地图资源事件。

```cpp
struct MapResourceEvent {
    MapResourceEventType type{MapResourceEventType::SCENES_UPDATED};
    std::string          scene_name;
    std::string          path_name;  // 仅路径相关事件有值
};
```

回调类型定义

类型	签名
`LocalizationStateEventCallback`	`std::function<void(const LocalizationState&)>`
`NavigationStateEventCallback`	`std::function<void(const NavigationState&)>`
`NavigationResultEventCallback`	`std::function<void(const NavigationResult&)>`

类型	签名
`PauseStateEventCallback`	`std::function<void(const PauseState&)>`
`MapResourceEventCallback`	`std::function<void(const MapResourceEvent&)>`
`MapTransferCallback`	`std::function<void(ErrorCode)>`

8.4 运动类型（motion/types.hpp）

GaitType {#gaittype}

机器人步态类型。

```
enum class GaitType {
    WALK = 0,  // 行走步态
    RUN = 1,   // 奔跑步态
};
```

MotionMode {#motionmode}

机器人运动模式。

```
enum class MotionMode : uint32_t {
    Manual = 0,       // 手动控制模式
    Assisted = 1,     // 辅助控制模式
    Navigation = 2,   // 导航模式
};
```

MotionState {#motionstate}

当前运动状态。

```
struct MotionState {
    GaitType gait{GaitType::WALK};
};
```

8.5 机器人类型（robot/types.hpp）

RobotOptions {#robotoptions}

创建机器人实例时的连接与心跳配置。

```
struct RobotOptions {
    std::string robot_ip;          // 机器人 IP 地址
    int         mqtt_port = 11058;  // MQTT 端口
};
```

LogLevel {#loglevel}

日志等级。

```
enum class LogLevel {
    DEBUG = 0,  // 调试日志
    INFO = 1,   // 信息日志
    WARN = 2,   // 警告日志
    ERROR = 3   // 错误日志
};
```

LogCallback {#logcallback}

日志输出回调。

```
using LogCallback = std::function<void(LogLevel,
                                       const std::string& logger_name,
                                       const std::string& message)>;
```

8.6 版本信息（version.hpp）

`robot/quad/version.hpp` 由构建系统根据 SDK 版本生成。业务代码可通过版本宏或 `robot::quad` 命名空间下的 `constexpr` 访问函数读取当前 SDK 版本。

版本宏

生成后的头文件会定义以下宏：

```
ROBOT_QUAD_VERSION_STRING
ROBOT_QUAD_VERSION_MAJOR
ROBOT_QUAD_VERSION_MINOR
ROBOT_QUAD_VERSION_PATCH
```

版本访问函数

```
inline constexpr const char* sdk_version_string() noexcept;
inline constexpr int sdk_version_major() noexcept;
inline constexpr int sdk_version_minor() noexcept;
inline constexpr int sdk_version_patch() noexcept;
```

函数	返回值
`sdk_version_string()`	SDK 版本字符串
`sdk_version_major()`	主版本号
`sdk_version_minor()`	次版本号
`sdk_version_patch()`	补丁版本号

附录：类型索引

按模块分组

模块	类型 / 接口
Robot	RobotOptions
日志	LogLevel、LogCallback
版本	`ROBOT_QUAD_VERSION_*`、`sdk_version_string()`、`sdk_version_major()`、`sdk_version_minor()`、`sdk_version_patch()`
导航	NavigationModule、NavigationState、NavigationStatus、NavigationTaskMode、NavigationOutputStatus、NavigationPoint、NavigationPath、NavigationTrajectory
定位	MappingModule、LocalizationState
资源	ResourceModule、MapResourceSnapshot、MapResourceEvent、MapResourceEventType、MapTransferOptions、SceneInfo、SceneDetail
运动	MotionMode、MotionState、GaitType
通用	Pose、Pose2D、Point3D、Quaternion、Subscription、SubscribeOptions、AsyncCommandId、AsyncCommandStatus、AsyncCommandResult、AsyncCommandCallback
错误	ErrorCode、ErrorItem、ErrorDescriptionLanguage

按字母顺序

类型 / 接口	所在章节
`AsyncCommandCallback`	8.1
`AsyncCommandId`	8.1
`AsyncCommandResult`	8.1
`AsyncCommandStatus`	8.1
`ErrorCode`	8.2
`ErrorDescriptionLanguage`	8.2
`ErrorItem`	8.2
`GaitType`	8.4
`LocalizationStateEventCallback`	8.3
`LocalizationState`	8.3
`LogCallback`	8.5

类型 / 接口	所在章节
`LogLevel`	8.5
`MapResourceEvent`	8.3
`MapResourceEventCallback`	8.3
`MapResourceEventType`	8.3
`MapResourceSnapshot`	8.3
`MapTransferCallback`	8.3
`MapTransferOptions`	8.3
`MotionMode`	8.4
`MotionState`	8.4
`NavigationOutputStatus`	8.3
`NavigationPath`	8.3
`NavigationPoint`	8.3
`NavigationState`	8.3
`NavigationStatus`	8.3
`NavigationTaskMode`	8.3
`NavigationTrajectory`	8.3
`PauseState`	8.3
`PauseStateEventCallback`	8.3
`Pose`	8.1
`Pose2D`	8.1
`Point3D`	8.1
`Quaternion`	8.1
`RobotOptions`	8.5
`SceneDetail`	8.3
`SceneInfo`	8.3
`sdk_version_major()`	8.6
`sdk_version_minor()`	8.6

类型 / 接口	所在章节
`sdk_version_patch()`	8.6
`sdk_version_string()`	8.6
`SubscribeOptions`	8.1
`Subscription`	8.1